LectureAssist:

An Agentic Multimodal AI Assistant that Turns Lectures into Adaptive Study Material

By

Iftikhar Ahmed — ID # 2121019642
Salman Shafi — ID # 2211386042
Jarinah Tasnim — ID # 2121026642

Supervised by

Dr. Adnan Areefen
Department of Electrical and Computer Engineering
North South University

Submitted in partial fulfillment of the requirements
of the degree of Bachelor of Science in Computer Science and Engineering

August 9, 2026

Department of Electrical and Computer Engineering
North South University

APPROVAL

Iftikhar Ahmed , Salman Shafi , and Jarinah Tasnim
from the Department of Electrical and Computer Engineering, North South University
have worked on the Senior Design Project titled:

"LectureAssist: An Agentic, Multimodal AI Assistant that Turns Lectures into Adaptive Study Material"

under the supervision of **Dr. Adnan Areefen** in partial fulfillment of the requirement for the degree of Bachelor of Science in Computer Science and Engineering and has been accepted as satisfactory.

Supervisor's Signature

.....

Dr. Adnan Areefen
Department of Electrical and Computer Engineering
North South University
Dhaka, Bangladesh.

Chairman's Signature

.....

Dr. Mohammad Abdul Matin
Professor
Department of Electrical and Computer Engineering
North South University
Dhaka, Bangladesh.

DECLARATION

It is to declare that this project is our original work. No part of this work has been submitted elsewhere, partially or entirely, for the award of any other degree or diploma. All project related information will remain confidential and shall not be disclosed without the formal consent of the project supervisor.

Relevant previous works presented in this report have been adequately acknowledged and cited. The plagiarism policy, as stated by the supervisor, has been maintained.

Students' Names & Signatures

1. **Iftikhar Ahmed**

- -

2. **Salman Shafi**

- -

3. **Jarinah Tasnim**

- -

Acknowledgements

This work would not have been possible without the input and support of many people over the last two trimesters. We would like to express our gratitude to everyone who contributed to it in some way or other.

First and foremost, we would like to thank our supervisor, **Dr. Adnan Areefen**, Department of Electrical and Computer Engineering, North South University, for his guidance throughout this project. His insistence that the evaluation be made rigorous that the generation prompts be shown rather than described, that chain-of-thought and program-of-thought baselines be compared directly against our agentic pipeline, and that question generation and answer generation be treated as two separate agents whose answers must be cross-checked fundamentally shaped this work, and turned it from a demonstration into a study.

Our sincere gratitude goes to the Department of Electrical and Computer Engineering, North South University, for providing the academic environment and the resources that made this project possible.

We are also thankful to the open-source community, whose freely available models and tools — Gemma 3, Ollama, Whisper, EasyOCR, Sentence-Transformers and FAISS — are the foundation on which this system is built, and without which a project of this scope could not have been undertaken without significant funding.

Last but not the least, we owe a great deal to our families, including our parents, for their unconditional love and immense emotional support.

Abstract

Recorded lectures and digital course material are now abundant, but the tools that help students convert that material into practice and self assessment are not. Existing study aids either require the student to paste in clean text by hand, or depend on paid cloud APIs that are impractical in bandwidth limited and cost-sensitive settings. This report presents **LectureAssist**, an end-to-end agentic AI system that turns a raw lecture video or an academic document into structured study and assessment material without any proprietary cloud service. The system couples a dual-channel extraction layer scene-adaptive optical character recognition that distinguishes slides, blackboards, whiteboards and on-screen text, combined with Whisper-based speech transcription to a multi-agent generation pipeline that plans, retrieves, generates, validates and refines five types of quiz questions, all served by locally hosted Gemma 3 models through the Ollama runtime on a single consumer-grade GPU. A retrieval augmented chat interface and an adaptive difficulty engine that tracks per topic performance complete the loop from raw media to personalised assessment. The system was evaluated on a large scale study in which multiple-choice questions were generated from a college-level calculus chapter across three difficulty levels, and scored by an independent, larger judge model on question quality, grounding, diversity and separately — *answer correctness*, using a second Answer Generator agent that re-solves each question without seeing the marked option. The work demonstrates that high quality, personalised educational AI is achievable on accessible hardware, and that question generators must be judged on answer correctness and diversity, not on fluency alone.

Table of Contents

List of Figures

List of Tables

Chapter 1

Introduction

This chapter motivates the project, states its purpose and contributions, and outlines the structure of the remainder of the report.

1.1 Background and Motivation

The proliferation of recorded lectures and digital course material has created a fundamental imbalance in self directed learning: students have more content than ever, but fewer tools with which to extract, consolidate and assess their understanding of it. A single semester course now routinely produces dozens of hours of recorded lectures, slide decks and reference chapters. Reviewing that material passively re-watching a video, re-reading a chapter is one of the least effective study strategies available. Decades of work in the learning sciences instead point to *retrieval practice*: repeatedly testing oneself on the material produces substantially more durable learning than re-exposure to it. The obstacle is supply. Producing good practice questions is slow, expert work, and the questions that do exist are concentrated at the end of textbook chapters, are not tied to what a particular instructor actually emphasised in class, and run out long before a student has mastered a weak topic.

Existing AI assisted study tools address this gap only partially. Text based chatbots require the student to manually transcribe or paste lecture content before they can help, which puts the burden of the hardest step getting the lecture out of the video and into text back on the student. Cloud dependent question generation services remove that burden, but they are inaccessible in bandwidth limited regions, they charge per token at a scale that makes routine practice expensive, and they require sending course material which may be copyrighted or institutionally restricted to a third party server. Crucially, none of these tools close the full loop from raw instructional media, through intelligent content understanding, to graded and personalised assessment without human intervention at each stage.

There is also a quality problem that is easy to overlook. A large language model asked to write a multiple choice question will almost always produce something that *reads* like

a good question. Whether the option it marks as correct actually *is* correct is a separate matter, and one that fluency based evaluation cannot detect. A study tool that confidently teaches a student the wrong answer is worse than no tool at all. Any serious system for automatic question generation therefore has to be evaluated on answer correctness, and on whether it keeps producing genuinely different questions rather than paraphrasing the same one, not merely on whether its output is well written.

1.2 Purpose and Goal of the Project

The goal of this project is to build and rigorously evaluate **LectureAssist**, a fully local, API free system that ingests a lecture video or an academic document and autonomously produces validated, difficulty controlled assessment material, together with a retrieval grounded chat interface over the same content.

The system is built around three ideas working in concert. First, a *dualchannel extraction* layer simultaneously processes the visual channel of a lecture using scene type adaptive OCR that distinguishes slides, blackboards, whiteboards and on-screen text, each of which needs different image preprocessing and its audio channel via Faster Whisper transcription, producing a unified, timestamped lecture timeline. Second, a *multi-agent generation pipeline* decomposes quiz creation into specialist agents for planning, retrieval, generation, validation and targeted refinement, enabling structured quality control at the level of the individual question. Third, an *adaptive difficulty engine* maintains per topic performance histories across sessions and steers question allocation toward a student's weak areas. The entire system runs on locally hosted Gemma 3 models through the Ollama runtime, requires no external API keys, and is packaged for single GPU deployment on Google Colab with Google Drive session persistence.

The key contributions of this work are:

- A unified multi modal lecture understanding pipeline that fuses scene adaptive OCR with filtered speech transcription into a synchronised content timeline, supporting both video and document inputs.
- A multi-agent quiz architecture implementing a Plan → Retrieve → Generate → Validate → Refine cycle across five question types (MCQ, True/False, Fill-in-the-Blank, Short Answer and Flashcard), with per-question difficulty, grounding and instruction-compliance validation, and a validator that chooses *how* to remediate a rejected question rather than merely rejecting it.
- An adaptive assessment engine that tracks per-topic student performance across sessions and personalises question difficulty and distribution without manual configuration.
- A multi-tier grading system combining deterministic matching, fuzzy string similarity and LLM-based rubric evaluation for free-text short answers, with targeted

concept-level feedback on incorrect responses.

- A fully local, API-free deployment on consumer hardware using the Ollama inference runtime, with RAG-enabled semantic chat and persistent session storage.
- A large-scale evaluation harness that compares the agentic pipeline against three direct-prompting baselines — plain, chain-of-thought and program-of-thought — and which, unlike prior automatic question generation evaluations, separates *question quality* from *answer correctness* by having a second, larger Answer Generator agent independently re-solve every generated question without seeing the marked option.

1.3 Organization of the Report

The remainder of this report is organised as follows. Chapter 2 provides the background needed to follow the rest of the report — large language models and the prompting strategies used as baselines, retrieval-augmented generation, agentic architectures, speech recognition and OCR — then reviews the research literature and the existing commercial applications closest to this work, and identifies the gaps this project addresses. Chapter 3 describes the system design: the extraction layer, the agent architecture, the four question-generation pipelines with their prompts given verbatim, the two-agent answer cross-check, and the software components used to build it. Chapter 4 presents the evaluation: the environment, the metrics and exactly how each is computed, the large-scale multiple-choice results, and a discussion of what they show — including a failure mode of the agentic pipeline that the evaluation exposed. Chapter **??** discusses the societal, ethical and environmental implications and the design constraints they impose. Chapter **??** presents the project plan and budget. Chapter **??** maps the work onto the complex engineering problem and activity attributes. Chapter **??** summarises the project, states its limitations honestly, and sets out the work that follows from it.

Chapter 2

Background and Literature Review

This chapter presents the theoretical foundation and reviews the existing literature related to large language model based educational question generation. Recent advances in large language models have significantly improved the ability of intelligent systems to generate educational content, particularly multiple choice questions that support teaching, learning, and assessment. At the same time, ensuring that generated questions are pedagogically meaningful, cognitively appropriate, and aligned with educational objectives remains a challenging research problem. Consequently, recent studies have shifted their focus from simply generating fluent questions toward developing structured generation frameworks and comprehensive evaluation methodologies.

This chapter first introduces the fundamental concepts required to understand the proposed system, including large language models, prompting techniques, agent based architectures, and evaluation strategies for educational question generation. It then reviews recent research that investigates controlled question generation using large language models, with particular emphasis on hybrid multi agent generation frameworks and educational question evaluation benchmarks. Existing educational applications are subsequently discussed before identifying the research gaps that motivate the proposed work presented in this thesis.

2.1 Preliminaries

Educational question generation using large language models integrates concepts from natural language processing, prompt engineering, intelligent agent systems, and educational assessment. Understanding these foundational concepts is essential for designing systems that generate high quality assessment items while maintaining pedagogical relevance and consistency. This section introduces the key technologies and methodologies that underpin the proposed framework, including large language models, prompting strategies, agent based architectures, and evaluation techniques. These concepts provide the theoretical

foundation for the methodology presented in later chapters.

2.1.1 Large Language Models and Prompting

Large Language Models (LLMs) have emerged as one of the most significant advancements in artificial intelligence, demonstrating remarkable capabilities in understanding, reasoning over, and generating natural language. Built upon the Transformer architecture, LLMs are pretrained on extensive text corpora using self supervised learning, enabling them to capture complex linguistic patterns, semantic relationships, and contextual information. Through subsequent instruction tuning and alignment techniques, these models can perform a wide variety of language tasks including summarization, translation, question answering, content generation, and educational assistance without requiring task specific model architectures. Their versatility has led to widespread adoption across educational, industrial, and research applications, particularly in tasks that require generating coherent and contextually relevant textual content. [**?** **?**]

The emergence of LLMs has fundamentally changed the landscape of automatic educational question generation. Earlier approaches often relied on handcrafted linguistic rules or supervised neural models trained for a single task, limiting their adaptability across different educational domains. In contrast, modern LLMs can generate diverse assessment items from instructional materials using natural language instructions alone. This capability allows a single model to generate questions of different formats, difficulty levels, and cognitive requirements without extensive retraining. Consequently, recent research has increasingly focused on designing effective interaction strategies that enable language models to generate pedagogically meaningful and educationally appropriate assessment items rather than developing specialized generation models for each individual task. [**?** **?**]

The quality of responses produced by an LLM depends not only on the model itself but also on how the task is presented. Prompt engineering refers to the process of designing structured instructions that guide the model toward producing outputs aligned with specific objectives. In educational question generation, prompts commonly specify the source material, target knowledge concepts, question type, difficulty level, and expected output format. Well designed prompts reduce ambiguity, improve consistency, and help ensure that generated questions remain relevant to the learning objectives. Recent systems further enhance this process by dynamically constructing prompts using contextual information generated during intermediate processing stages, allowing the model to produce more accurate and context aware assessment items. [**?** **?**]

Several prompting techniques have been proposed to further improve educational question generation. One widely adopted approach is Chain of Thought prompting, which encourages the model to explicitly perform intermediate reasoning before generating the final output. Rather than producing a question immediately, the model first analyzes the instructional content, identifies important concepts, determines appropriate learning

objectives, and then constructs a question that reflects the intended educational outcome. This reasoning process improves the generation of higher order questions requiring conceptual understanding or logical inference while reducing inconsistencies in the generated assessment items. Modern educational generation frameworks frequently employ Chain of Thought prompting during planning and evaluation stages to improve both reliability and pedagogical quality. [**?**]

Another important prompting strategy is persona based prompting, where the language model is instructed to assume the role of an experienced teacher, curriculum designer, or assessment specialist. Assigning a professional role encourages the model to generate questions that more closely resemble those created by human educators while adhering to established educational practices. Recent studies also incorporate self critique prompting, which introduces an internal evaluation stage after question generation. During this stage, the language model reviews its own output by examining question clarity, answer correctness, distractor quality, and alignment with instructional objectives before revising the question if necessary. Together, these prompting strategies improve the controllability, consistency, and educational quality of modern large language model based question generation systems. [**?**]

2.1.2 Agent Based Architectures

Early large language model based educational question generation systems primarily relied on a single prompt to transform instructional content into assessment questions. Although these systems demonstrated impressive language generation capabilities, they often struggled to maintain consistency, accurately control cognitive complexity, and verify the correctness of generated questions. A single language model was expected to understand the source material, identify important concepts, generate meaningful questions, construct plausible distractors, and evaluate the quality of the final output within a single interaction. As the complexity of educational tasks increased, this monolithic approach frequently produced questions with ambiguous wording, incorrect answers, or distractors that failed to effectively distinguish different levels of student understanding. Consequently, recent research has shifted towards agent based architectures that decompose the overall generation process into multiple coordinated subtasks.

An agent based architecture consists of several specialized agents that collaborate to accomplish a complex task. Rather than assigning every responsibility to a single language model interaction, each agent performs a dedicated function while exchanging information with other agents throughout the workflow. Individual agents may be responsible for analyzing instructional content, identifying learning objectives, generating questions, evaluating assessment quality, or formatting the final output. This modular design improves transparency, allows intermediate outputs to be verified before proceeding to subsequent stages, and enables individual components to be refined independently without affecting the entire system. Such characteristics make agent based architectures particularly suit-

able for educational applications where accuracy, consistency, and pedagogical quality are essential.

A representative example of this approach is the ReQUESTA framework, which employs a hybrid multi agent architecture for automatic multiple choice question generation. Instead of relying solely on a language model, the framework combines deterministic rule based components with LLM powered reasoning agents. Rule based modules perform structured operations such as document segmentation, workflow coordination, and output formatting, while specialized language model agents perform higher level reasoning tasks including concept identification, question generation, answer verification, and quality evaluation. This hybrid design leverages the reliability of traditional algorithms together with the flexibility and reasoning capability of modern large language models, resulting in more robust educational question generation.

Within the ReQUESTA workflow, the instructional material is first divided into manageable text segments before being analyzed by a planning agent. The planning agent identifies important concepts, determines suitable learning objectives, and prepares a structured generation plan for subsequent stages. Based on this plan, specialized generation agents create different categories of questions that target various cognitive skills. A dedicated evaluation agent then reviews each generated question by examining its clarity, correctness, and educational relevance. If deficiencies are identified, the question is returned to the corresponding generation agent for revision, creating an iterative refinement process that continues until the generated assessment satisfies predefined quality requirements. Finally, formatting components standardize the presentation of the questions for learners.

The adoption of agent based architectures demonstrates that improvements in educational question generation depend not only on increasingly capable language models but also on carefully designed workflows that coordinate reasoning, generation, and evaluation. By separating complex educational tasks into specialized components, these architectures provide greater control over question quality, improve alignment with learning objectives, and facilitate the development of scalable educational assessment systems. Consequently, agent based workflows have become an important direction in recent research on large language model based educational question generation.

2.1.3 Evaluation of Educational Question Generation

Evaluating automatically generated educational questions is considerably more challenging than evaluating general text generation tasks. While traditional natural language generation metrics such as BLEU, ROUGE, and METEOR measure lexical similarity between generated text and reference answers, they often fail to capture the educational quality of assessment items. Multiple valid questions can be generated from the same instructional material using different wording, reasoning processes, or assessment formats while remaining equally effective for learning. Consequently, relying solely on text sim-

ilarity metrics may incorrectly penalize high quality questions that differ from reference examples, making conventional evaluation methods insufficient for educational question generation.

Recent research has therefore shifted towards pedagogically oriented evaluation frameworks that assess whether generated questions satisfy educational objectives rather than merely matching reference text. Important evaluation criteria include the correctness of the generated answer, alignment between the question and the intended knowledge point, appropriateness of the question type, clarity of language, cognitive complexity, and the quality of distractors in multiple choice questions. These dimensions provide a more comprehensive assessment of educational usefulness by considering both linguistic quality and instructional effectiveness.

One notable contribution in this area is EQGBench, a benchmark specifically developed for evaluating educational question generation using large language models. Unlike conventional benchmarks, EQGBench introduces a multidimensional evaluation framework that measures educational quality from several complementary perspectives. The benchmark evaluates knowledge point alignment, question type alignment, overall question quality, solution quality, and competence oriented guidance. Together, these dimensions assess whether generated questions accurately reflect instructional content, employ appropriate assessment formats, provide correct and complete solutions, and promote meaningful learning outcomes rather than simple factual recall. This multidimensional framework offers a more reliable assessment of educational question generation than traditional text similarity metrics alone. [**?**]

Another important advancement is the use of large language models as evaluators. Instead of depending exclusively on automatic similarity metrics or manual expert review, recent systems employ language models to assess generated questions according to predefined educational criteria. Acting as impartial evaluators, these models examine question clarity, factual correctness, relevance to the source material, distractor plausibility, and overall pedagogical quality. This approach enables scalable evaluation while maintaining consistency across large numbers of generated assessment items. Human expert evaluation remains the gold standard for educational assessment; however, LLM based evaluation has emerged as a practical alternative for benchmarking and iterative system development.

The increasing emphasis on comprehensive evaluation reflects the growing maturity of educational question generation research. As generation quality continues to improve through advances in large language models and agent based workflows, evaluation methodologies have likewise evolved to measure educational effectiveness rather than linguistic similarity alone. These developments provide a stronger foundation for designing intelligent assessment systems capable of generating reliable, pedagogically meaningful, and cognitively appropriate questions for diverse educational settings. [**? ?**]

2.2 Literature Review

Automatic educational question generation has been an active area of research for several decades, evolving alongside advances in natural language processing and artificial intelligence. Early approaches relied primarily on manually designed linguistic rules and handcrafted templates to transform declarative sentences into questions. Although these methods were computationally efficient and generated grammatically correct questions within restricted domains, they required extensive human effort and exhibited limited adaptability to diverse educational materials. As machine learning techniques matured, researchers gradually shifted towards data driven approaches capable of learning question generation patterns directly from large annotated datasets. More recently, the emergence of large language models has fundamentally transformed the field by enabling the generation of high quality, context aware, and pedagogically meaningful assessment items through natural language instructions rather than task specific model architectures.

Despite these advances, producing educational questions that accurately assess learning outcomes remains a challenging problem. High quality assessment items must not only be grammatically correct but also align with instructional objectives, target appropriate cognitive levels, contain unambiguous answers, and provide effective distractors for multiple choice questions. Consequently, recent research has increasingly focused on developing structured generation frameworks and comprehensive evaluation methodologies that improve both the quality and educational value of automatically generated questions.

This section reviews the major developments in educational question generation, beginning with traditional approaches before discussing recent large language model based methods. Particular emphasis is placed on two representative studies that address complementary aspects of the problem: the ReQUESTA framework, which proposes a hybrid multi agent architecture for controlled multiple choice question generation, and EQG-Bench, which introduces a comprehensive benchmark for evaluating the educational quality of questions generated by large language models. Together, these studies represent important advancements in both the generation and evaluation of intelligent educational assessment systems.

2.2.1 Evolution of Educational Question Generation

The development of automatic educational question generation has closely followed the evolution of natural language processing techniques. Early research primarily relied on rule based systems that transformed declarative sentences into interrogative forms using handcrafted linguistic rules, syntactic parsing, and manually designed templates. These approaches generated grammatically correct questions within restricted domains and offered high interpretability because the generation process was explicitly defined by human experts. However, developing and maintaining large collections of language rules required substantial manual effort, while the resulting systems often struggled to generalize across different subjects, writing styles, and educational contexts. As a result, rule based meth-

ods exhibited limited scalability and were unsuitable for generating diverse assessment items from complex instructional materials.

The availability of large educational datasets and advances in machine learning led researchers to adopt data driven approaches for question generation. Sequence-to-sequence neural networks enabled models to learn mappings between instructional content and corresponding questions directly from annotated examples rather than relying on manually designed rules. Attention mechanisms further improved these models by allowing them to focus on the most relevant portions of the input text during generation. Compared with traditional rule based systems, neural approaches produced more fluent and diverse questions while requiring significantly less manual feature engineering. Nevertheless, their performance depended heavily on the availability of large, high quality training datasets and they often struggled to generate questions requiring complex reasoning or deep conceptual understanding.

The introduction of Transformer based architectures marked another major advancement in automatic question generation. Transformer models demonstrated superior contextual understanding through self attention mechanisms, enabling them to capture long range dependencies and semantic relationships more effectively than recurrent neural networks. Pretrained language models such as BERT, T5, and BART further improved generation quality by leveraging knowledge acquired from large scale pretraining before being adapted to educational tasks. These models significantly enhanced grammatical fluency, contextual relevance, and the diversity of generated questions. However, most Transformer based systems still required task specific fine tuning and were generally designed for fixed question generation objectives, limiting their flexibility across different educational settings.

Recent progress in large language models has fundamentally transformed educational question generation. Unlike earlier approaches that required dedicated training for individual tasks, modern LLMs can generate assessment items through carefully designed natural language prompts without extensive task specific optimization. This capability enables a single model to produce multiple question formats, adjust difficulty levels, and incorporate reasoning processes using prompt engineering techniques alone. Contemporary research has therefore shifted its emphasis from improving language generation itself toward developing structured workflows, agent based architectures, and comprehensive evaluation methodologies that ensure generated questions satisfy pedagogical objectives. These developments represent a transition from purely language focused generation toward intelligent educational assessment systems capable of producing reliable, context aware, and pedagogically meaningful questions across diverse learning environments. [**?** **?**]

2.2.2 Large Language Model Based Educational Question Generation

The emergence of large language models has significantly advanced the field of educational question generation by enabling the production of coherent, context aware, and pedagog-

ically relevant assessment items without extensive task specific training. Unlike earlier neural approaches that required fine tuning for individual datasets or question types, modern LLMs leverage knowledge acquired during large scale pretraining to perform educational tasks through carefully designed natural language prompts. This capability has greatly improved the adaptability of question generation systems, allowing them to operate across multiple academic disciplines and educational settings while requiring only minimal modifications to the input instructions.

Recent research has increasingly focused on improving the controllability of question generation rather than solely enhancing linguistic quality. One representative study is the ReQUESTA framework, which proposes a hybrid multi agent architecture for generating cognitively diverse multiple choice questions. The framework recognizes that educational assessment involves several interconnected tasks, including identifying learning objectives, selecting appropriate concepts, constructing meaningful questions, generating plausible distractors, and verifying the quality of the final assessment item. Instead of assigning all these responsibilities to a single language model interaction, ReQUESTA distributes them among multiple specialized agents that collaborate throughout the generation process. This modular design enables greater control over each stage of question generation while reducing errors that commonly occur in single prompt systems. [**?**]

The ReQUESTA framework combines deterministic rule based processing with the reasoning capability of large language models. Rule based components manage operations such as document segmentation, workflow coordination, and output formatting, while specialized LLM agents perform higher level reasoning tasks including concept identification, question generation, quality evaluation, and revision. The workflow begins by dividing instructional material into manageable text segments, after which a planning agent analyzes each segment to identify key concepts and determine suitable learning objectives. Based on this analysis, dedicated generation agents create multiple choice questions targeting different cognitive skills before an evaluation agent reviews each question for correctness, clarity, and educational relevance. Questions that do not satisfy predefined quality criteria are automatically revised through an iterative feedback process until acceptable quality is achieved. This structured workflow improves both the reliability and consistency of generated assessment items compared with conventional single prompt approaches. [**?**]

Another important contribution of ReQUESTA is its use of advanced prompting strategies throughout the generation pipeline. The framework incorporates Chain of Thought reasoning to encourage systematic analysis before question generation, persona based prompting to emulate experienced educators during assessment design, and self critique mechanisms that enable the language model to evaluate and refine its own outputs. By integrating these techniques within a coordinated multi agent workflow, the framework produces questions with clearer wording, more accurate answers, and higher quality distractors. Experimental results reported in the study demonstrate that this hybrid architecture consistently outperforms conventional zero shot prompting approaches in both automated evaluation and expert assessment, highlighting the importance of structured

workflows in educational question generation. [**?**]

The findings of the ReQUESTA study demonstrate that advances in educational question generation depend not only on increasingly capable language models but also on carefully designed system architectures that coordinate planning, reasoning, generation, and evaluation. These observations have influenced recent research by encouraging the development of modular, collaborative frameworks that improve both the pedagogical quality and reliability of automatically generated educational assessments.

2.2.3 Evaluation Frameworks for Educational Question Generation

While recent advances in large language models have significantly improved the generation of educational questions, evaluating the quality of these questions remains an equally important challenge. Unlike conventional natural language generation tasks, educational question generation cannot be assessed solely through lexical similarity between generated and reference texts. Multiple questions may correctly assess the same learning objective despite differing in wording, structure, or reasoning process. Consequently, traditional evaluation metrics such as BLEU, ROUGE, and METEOR often fail to reflect the true pedagogical value of generated assessment items. This limitation has motivated researchers to develop evaluation frameworks that measure educational effectiveness rather than textual similarity alone.

A notable contribution in this direction is EQGBench, a benchmark specifically designed for evaluating educational question generation using large language models. The benchmark was developed to address the absence of standardized evaluation methodologies capable of measuring both linguistic quality and educational relevance. Rather than relying exclusively on automatic similarity metrics, EQGBench introduces a multidimensional evaluation framework that examines several complementary aspects of educational assessment. These dimensions include knowledge point alignment, question type alignment, question quality, solution quality, and competence oriented guidance. Together, these criteria provide a comprehensive assessment of whether generated questions accurately reflect instructional content while supporting meaningful learning outcomes. [**?**]

Knowledge point alignment evaluates whether a generated question correctly targets the intended concepts presented in the instructional material. Question type alignment determines whether the generated assessment follows the desired format, such as multiple choice, short answer, or explanatory questions. Question quality focuses on linguistic clarity, logical consistency, and the absence of ambiguity, while solution quality measures the correctness and completeness of the accompanying answers or explanations. Competence oriented guidance extends the evaluation beyond factual correctness by assessing whether the generated questions encourage reasoning, conceptual understanding, and higher order thinking skills. Collectively, these dimensions provide a richer understanding of educational quality than conventional text based evaluation metrics. [**?**]

To validate the proposed benchmark, the authors constructed a comprehensive evaluation dataset consisting of educational materials from mathematics, physics, and chemistry. The benchmark was then used to evaluate numerous state of the art large language models under a standardized experimental protocol. The results demonstrated substantial variation in educational performance among different models despite many of them achieving comparable scores on traditional language generation benchmarks. These findings highlight that strong general language capabilities do not necessarily translate into high quality educational assessment generation, emphasizing the importance of specialized evaluation frameworks tailored to educational applications. [**?**]

Another significant contribution of recent evaluation research is the growing adoption of large language models as automated evaluators. Instead of depending exclusively on human experts or lexical similarity metrics, language models can assess generated questions according to predefined educational criteria such as clarity, correctness, relevance, and pedagogical appropriateness. Although human evaluation remains the most reliable method for assessing educational quality, LLM based evaluation offers a scalable and consistent alternative for benchmarking large numbers of generated questions during system development. The integration of comprehensive evaluation frameworks such as EQGBench with automated LLM based assessment represents an important step toward developing reliable, standardized methodologies for educational question generation research. [**?**]

2.3 Research Gap

The literature demonstrates that recent advances in large language models have significantly improved the generation and evaluation of educational questions. The ReQUESTA framework shows that hybrid multi agent architectures can produce more reliable and cognitively diverse multiple choice questions by separating planning, generation, evaluation, and refinement into specialized components. Similarly, EQGBench establishes that evaluating educational question generation requires multidimensional assessment criteria that consider pedagogical quality in addition to traditional language generation metrics. Together, these studies represent important progress toward intelligent educational assessment systems.

Despite these contributions, several research challenges remain. Existing generation frameworks primarily focus on producing high quality assessment items from textual content but provide limited support for integrating diverse educational resources into a unified generation pipeline. Many systems assume that the instructional content has already been prepared in a structured textual format, making them less suitable for practical educational environments where learning materials may originate from lecture notes, presentation slides, or other unstructured documents.

Furthermore, although recent studies emphasize question quality and evaluation, relatively little attention has been given to developing end to end educational assistance systems that combine document understanding, intelligent question generation, automated

answer production, and comprehensive evaluation within a single workflow. In many existing approaches, these components are investigated independently rather than being integrated into a cohesive educational framework capable of supporting both instructors and learners.

Another limitation is that current research predominantly evaluates generated questions using benchmark datasets under controlled experimental settings. While these evaluations provide valuable insights into model performance, they may not fully reflect real world educational scenarios involving diverse instructional materials, varying document structures, and different learning objectives. Consequently, there remains a need for practical systems that effectively combine large language models with structured generation workflows while maintaining high educational quality across different types of learning resources.

Motivated by these research gaps, this thesis proposes a large language model based educational question generation framework that integrates document processing, intelligent question generation, answer generation, and automated evaluation into a unified system. The proposed approach aims to generate contextually relevant and pedagogically meaningful assessment items while providing a flexible and scalable workflow that can be applied to a wide range of educational materials.

Chapter 3

Methodology

This chapter describes how LectureAssist is built: the overall system design and the flow of data through it, the software components chosen and why, and the implementation of each stage — extraction, retrieval, the multi-agent generation loop, the four question-generation pipelines whose prompts are given verbatim, the two-agent answer cross-check, and the grading and adaptation layers.

3.1 System Design

LectureAssist is organised as a pipeline of specialist agents coordinated by an orchestrator. The user supplies either a lecture video or a document (PDF, DOCX, PPTX); the system converts it into a single grounded content representation, and every downstream capability — summary, quiz, chat, adaptive practice — consumes that representation rather than the raw media. The design is deliberately staged so that each stage produces an artifact that can be inspected, cached and re-used, which is what makes the expensive stages (OCR, transcription) survivable on free-tier hardware.

3.1.1 Data flow

The end-to-end flow is as follows.

1. **Ingestion.** A video is sampled into frames and its audio track is demuxed; a document is parsed page by page.
2. **Dual-channel extraction.** The visual channel goes to the *Extractor* agent (scene-adaptive OCR); the audio channel goes to the *Transcriber* agent (Faster-Whisper). Documents bypass both and go to the *DocParser* agent.
3. **Fusion.** OCR blocks and transcript segments are merged in timestamp order into a single unified lecture timeline, so that what the lecturer *said* sits next to what they *wrote* at the same moment.

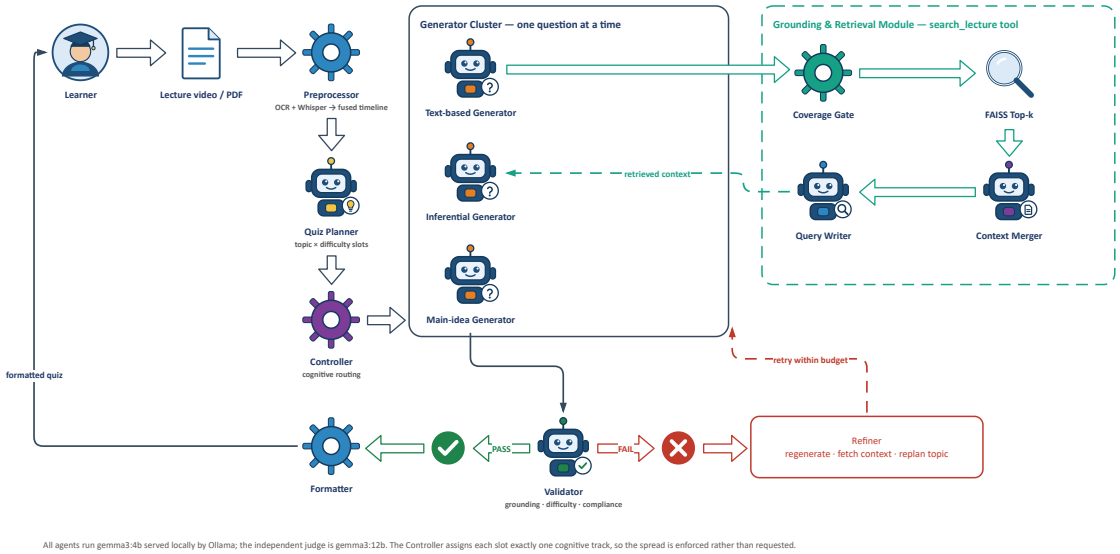

Figure 3.1: System architecture of LectureAssist. The orchestrator coordinates specialist agents across ingestion, representation and generation: the Controller routes each slot to one of three cognitive tracks, the Validator accepts or rejects each drafted question, and a rejected question is diagnosed and returned to the Refiner rather than discarded. The dashed module on the right is the model-directed `search_lecture` retrieval tool.

4. **Understanding.** The *Summary* agent produces a structured summary; the *RAG-Builder* agent chunks the content, embeds it and builds a vector index; the *Hints* agent flags passages the lecturer marked as exam-relevant.

5. **Generation.** The *Quiz Planner* allocates questions across topics and difficulties; a *Question Generator* drafts one question at a time; a *Validator* accepts or rejects it and chooses a remediation action; a *Refiner* applies it. This loop is the core of the system and is described in Section 3.3.1.

6. **Assessment and adaptation.** Answers are graded by a multi-tier grader; per-topic performance is written to a persistent profile, which the *Adaptive Quiz* agent uses to bias the next quiz toward weak topics.

3.1.2 Scene-adaptive extraction

A single OCR preprocessing recipe cannot handle a real lecture. White chalk on a dark board is a low-contrast, inverted-polarity image; a projected slide in a lit room is a bright, high-contrast one; a dark-theme slide deck looks superficially like a blackboard but is destroyed by the inversion that a blackboard requires. LectureAssist therefore *classifies each sampled frame first* and preprocesses it accordingly. Frames are routed by brightness and edge statistics into four classes — `slide`, `blackboard`, `whiteboard` and generic `scene` (on-screen text) — and each class gets its own contrast enhancement, thresholding and, where appropriate, polarity inversion before being passed to EasyOCR. Consecutive frames whose extracted text is more than 78% similar are treated as the same board state and

deduplicated, so that a slide left on screen for two minutes contributes one block rather than sixty.

3.1.3 Retrieval

The fused timeline is chunked, embedded with the Sentence-BERT model `all-MiniLM-L6-v2` [**?**], L_2-normalised, and indexed in FAISS [**?**] with an inner-product index, so that inner product equals cosine similarity (Eq. **??**). Retrieval returns the top $k = 5$ chunks together with their timestamps, which is what allows both the chat interface and a generated question to be traced back to the moment in the lecture they came from. Crucially, retrieval is exposed to the generator agent as a *tool* (`search_lecture`) rather than being applied unconditionally: the agent decides for itself whether it needs more evidence.

3.2 Hardware and/or Software Components

The system is written in Python and served as a Flask API, with a single-page HTML/JavaScript front end that talks to it over an ngrok tunnel; this split lets the heavy models run on a Colab GPU while the interface runs in the student's own browser. Table 3.1 lists the components, the alternatives considered, and the reason for each choice. The overriding constraint throughout was that *nothing may require a paid API key*, since the target users are precisely those for whom a per-token bill is prohibitive.

Table 3.1: Software components, alternatives, and selection rationale.

Tool	**Function**	**Other similar tools**	**Why selected**
Gemma 3 (4B / 12B)	Generation, summarisation, grading, judging	GPT-4, Claude, Llama 3, Mistral	Open weights, runs locally, no API key; two sizes let the cheap model generate and the strong model judge.
Ollama	Local LLM serving	llama.cpp, vLLM, HF Transformers	One-line model pull, automatic GPU offload, OpenAI-style local HTTP API; vLLM needs more VRAM than a free T4 has.
Faster-Whisper (`small`)	Speech-to-text	OpenAI Whisper API, Vosk, wav2vec2	Same accuracy as reference Whisper at a fraction of the memory; runs offline. The API version is paid and cloud-bound.
EasyOCR (en, bn)	Board and slide text extraction	Tesseract, PaddleOCR, Google Vision	Deep-learning based, far better on handwriting than Tesseract; native Bengali support matters for local lectures; Google Vision is a paid cloud API.
Sentence-Transformers `all-MiniLM-L6-v2`	Embeddings for RAG and for evaluation metrics	OpenAI `text-embedding-3`, BGE, E5	384-dim, small and fast enough to co-exist with the LLM on one GPU; strong quality-per-byte; local.
FAISS	Vector index	Chroma, Pinecone, pgvector	In-process, no server, no network; exact inner-product search is more than sufficient at lecture scale.
PyMuPDF	PDF/DOCX/PPTX parsing	pdfplumber, PyPDF2	Fastest reliable text-layer extraction; preserves page structure used for per-page source attribution.
Flask + ngrok	Backend API and public tunnel	FastAPI, Gradio, Streamlit	Minimal surface for a JSON API; ngrok exposes the Colab GPU to a browser UI running on the student's own machine.
SymPy	Building the verified answer key for calibration	Hand-computed answers, WolframAlpha	Answers are *machine-computed*, not hand-typed, so the calibration set cannot inherit a human arithmetic error.
Google Colab (T4) + Drive	Compute and persistence	Local GPU, RunPod, AWS	Free tier is the realistic deployment target; Drive persistence lets a run survive a runtime disconnect.

3.3 Hardware and/or Software Implementation

3.3.1 The agentic question-generation loop

The agentic Question Generator produces *one question at a time* through a multi-stage loop. The design decision that distinguishes it from a generate-then-filter pipeline is that the validator does not merely reject a bad question — it *diagnoses* it and chooses how the next attempt should differ. The stages are:

1. **Plan.** The Quiz Planner reads the lecture summary and distributes the requested questions across the topics that are actually present, each with a target difficulty, yielding a list of (topic, difficulty) slots.

2. **Retrieve.** Before drafting a slot, the generator inspects the content it holds and decides for itself whether it has enough material on that topic. If not, it issues its own retrieval query through the `search_lecture` tool to pull more context. This is genuine tool use: the decision to retrieve is the model's, not a fixed step in the pipeline.

3. **Generate.** It drafts exactly one question under a grounding rule — the question must be answerable using only the supplied content — rotating a question strategy on each attempt (concept-check, scenario, compare-and-justify) so that a retry is not merely a resample of the same prompt.

4. **Validate.** A separate call checks the draft on three axes — difficulty match, grounding, and instruction compliance — and returns `PASS` or `FAIL` together with a remediation `action`.

5. **Retry (loop back).** On `FAIL` the loop repeats within a retry budget, and the validator's chosen action decides the fix: `regenerate` (rewrite, with the failure reason fed back into the prompt), `fetch_context` (retrieve more evidence first, then retry), or `replan_topic` (abandon a topic that is not really in the lecture and swap to one that is).

6. **Override.** If the retry budget is exhausted, the best attempt so far is kept and explicitly *labelled* as an override. It is never silently recorded as a clean pass — otherwise the pipeline's own accept rate would be a fiction.

3.3.2 The four generation pipelines

To isolate the contribution of the agentic machinery, the same content is used to generate questions through four pipelines that differ *only* in how the model is asked:

- **Agentic** — the Plan → Retrieve → Generate → Validate → Retry loop of Section 3.3.1.
- **Non-agentic (plain)** — a single LLM call that emits the questions directly: no planning, no retrieval, no validation, no retry.
- **Chain-of-thought (CoT)** — a single call in which the model must reason step by step to *derive* each answer before committing to it, and record that reasoning.
- **Program-of-thought (PoT)** — a single call in which the model must derive each answer through an explicit, ordered sequence of computation steps (define, substitute, simplify, evaluate) and mark the option equal to the final computed value.

CoT and PoT are both *single-shot and non-agentic*: one LLM call, no planning, retrieval, validation or retry. The only thing that separates them from the plain baseline is the wording of the prompt. They emit the same JSON schema as the plain generator, plus one extra field (`reasoning` or `computation`) that is retained for inspection but ignored by the downstream judge and Answer Generator, so all four pipelines feed into an identical scoring path and are directly comparable. The two prompts are reproduced verbatim below, since the prompt *is* the method.

Chain-of-thought generation prompt

```
Generate [N] high-quality MCQ from this
lecture content.
Difficulty focus: [DIFF]
Use CHAIN-OF-THOUGHT: for EACH question,
think step by step to work out the correct
answer BEFORE choosing it, and record that
step-by-step reasoning. The option you mark
correct MUST be exactly the answer your
reasoning reaches, and each distractor must
be wrong for a stateable reason.
STRICT JSON: {"questions":[{"question",
  "options":{"A","B","C","D"},
  "reasoning","correct_answer",
  "explanation","topic","difficulty",
  "bloom_level","source_timestamp",
  "source_type"}]}
'reasoning' = your step-by-step
derivation of the correct option.

CONTENT: [up to 8000 chars]
SUMMARY: [up to 1500 chars]
```

Program-of-thought generation prompt

```
Generate [N] high-quality MCQ from this
lecture content.
Difficulty focus: [DIFF]
Use PROGRAM-OF-THOUGHT: for EACH question,
derive the answer through an explicit
ordered sequence of computation/derivation
steps (like a short program: define,
substitute, simplify, evaluate), record
those steps, and only then mark the option
that EQUALS the final computed result. The
marked correct option MUST equal the
endpoint of your computation.
STRICT JSON: {"questions":[{"question",
  "options":{"A","B","C","D"},
  "computation","correct_answer",
```

```
  "explanation","topic","difficulty",
  "bloom_level","source_timestamp",
  "source_type"}]}
'computation' = your ordered derivation
steps ending at the correct value.

CONTENT: [up to 8000 chars]
SUMMARY: [up to 1500 chars]
```

3.3.3 Answer correctness: two agents that must agree

A question can be well written and still mark the wrong option. Question generation and answer generation are therefore treated as *two distinct agents*, and their answers are cross-checked. Correctness is measured three complementary ways:

- **Independent re-solve (Answer Generator agent).** The larger model is given the stem and the options *with the marked answer removed*, and solves the question from scratch. Its choice is compared with the Question Generator's marked answer; the agreement rate is the *independent-match rate*. The solver never sees what it is supposed to agree with.
- **Judge-verify.** While scoring quality, the judge is additionally asked whether the marked answer is actually correct given the content, yielding the *judge-verified-correct rate*. This is free — it rides along on a call that was already being made.
- **Calibration on a known-answer set.** The generated questions have no ground-truth key, so neither of the above is an absolute measure of correctness — if generator and solver share a misconception, they agree and are both wrong. To bound this, both models independently solve a set of MCQ whose answers *are* known, and their accuracy on it calibrates how much the agreement rate above is worth. The calibration set is stratified into *computational* items, whose answers are computed with SymPy [**?**] rather than typed by hand, and *conceptual* items drawn from the chapter's definitions and theorems. This is reported separately and is explicitly *not* a score on the generated questions.

The calibration set is used for evaluation only. It is never placed on the generation path, since a generator that had seen the answer key would be scored on questions it had effectively been given, and the resulting numbers would be contaminated.

3.3.4 Multi-tier grading and adaptation

Student answers are graded by a cascade chosen to match the question type. MCQ and True/False are graded by exact key match. Fill-in-the-Blank is graded by a fallback chain of exact match, then containment, then fuzzy string similarity above a 0.80 threshold, so that a correct answer is not marked wrong over a typo or a plural. Short Answer, where

no string comparison is adequate, is graded by the 12B model against a rubric, which also returns concept-level feedback naming what the student missed rather than merely marking it incorrect.

Every graded response updates a persistent per-topic profile. The Adaptive Quiz agent reads that profile and biases the next quiz's topic and difficulty allocation toward the topics where the student's accuracy is lowest, so that practice concentrates where it is needed instead of being uniformly distributed.

Chapter 4

Investigation/Experiment, Result, Analysis and Discussion

This chapter reports the evaluation of LectureAssist's question generator. It describes the environment the experiment was run in, enumerates the configurations under test, defines the evaluation instruments precisely, and reports the results that have been measured — marking clearly, and in every table, those that have not.

The evaluation has two halves. The first asks whether the agentic pipeline produces better multiple-choice questions than single-shot prompting, judged under *two published protocols* run side by side: **EQGBench**, a benchmark for evaluating an LLM's ability to *generate* educational questions, and **ReQUESTA**, an agentic MCQ-authoring framework whose expert rubric provides a second, independent instrument. The two are reported separately and never merged, because they use different scales and different scoring rules (Section 4.2). A third set of automated metrics, native to this project, is reported alongside them.

The second half is new to this chapter and is the direct consequence of the first half's outcome. Because the completed measurements show the agentic pipeline *failing* to beat its own baseline (Section 4.4.3), the interesting question is no longer "does the agentic loop win" but "*which agent* is responsible, and how much does each one cost". Two families of controlled variants answer that: **leave-one-agent-out** ablations, which delete one agent at a time (Section 4.3.1), and **agent-merging** variants, which collapse two agents into a single LLM call (Section 4.3.2).

The experiment is designed around five questions. *RQ1:* judged on each published protocol, does the agentic pipeline produce better multiple-choice questions than single-shot prompting? *RQ2:* how does each pipeline behave as the target difficulty rises from easy to hard? *RQ3:* which dimensions actually separate the pipelines, and which saturate? *RQ4:* which individual agent accounts for the agentic pipeline's behaviour — in particular, is the retry loop the cause of its excess repetition? *RQ5:* can two agents be merged into one call without losing quality, and how much compute does that save?

4.1 Environment Setup

Question *generation* was run on a single Google Colab instance with an NVIDIA T4 GPU (16 GB), with models served locally by Ollama; no paid or proprietary API is used anywhere in the generation path. Bulk generation artifacts are written to Google Drive so a run survives a Colab disconnection and can be resumed rather than restarted.

4.1.1 Configurations under test

Eleven configurations are defined. All share the same pinned `gemma3:4b` generator, the same source material, the same retrieval tool and the same cognitive-routing Controller; they differ only in which agents are present and how those agents are packaged into LLM calls. Holding everything else fixed is what makes any difference attributable to the manipulation rather than to a confound.

Table 4.1: The eleven configurations, what each manipulates, and its implementation/measurement status at the time of writing.

Configuration	Manipulation	Question it answers	Status
Baseline comparison (RQ1–RQ3)			
`agentic`	full loop: Plan → Retrieve → Generate → Validate → Retry	control	scored
`nonagentic`	one LLM call, no planning, retrieval, validation or retry	does the machinery help?	scored
`cot`	single call, chain-of-thought prompt	can a prompt substitute?	unscored
`pot`	single call, program-of-thought prompt	can a prompt substitute?	unscored
Leave-one-agent-out ablations (RQ4)			
− planner	uniform slots at the target difficulty; no Quiz Planner	does planning help, or cause topic tunnel-vision?	not built
− retrieval	fixed content snippet; `search_lecture` disabled	is model-directed RAG earning its cost?	not built
− validator	first draft accepted; no validation, no retry	the whole quality gate	not built
− refiner	validator still runs and labels, but never remediates	**separates detection from remediation**	not built
− routing	`COGNITIVE_ROUTING=False`	does the Controller buy diversity?	not built
Agent-merging variants (RQ5)			
`merge_vr`	Validator + Refiner in ONE call: it judges the draft and returns the rewrite in the same reply	is a separate refiner call worth it?	built, unrun
`merge_pg`	Planner + Generator in ONE call: the generator picks its own topic and drafts the question	is a separate planner call worth it?	built, unrun

Scope of the results reported in this chapter. Of the eleven configurations, **only `agentic` and `nonagentic` have completed a scored evaluation run** (Section 4.4.3). The `cot` and `pot` pipelines are implemented but have not been judged. The two merging variants are implemented in a standalone harness (`merged_agents.py`) and verified, but have not been run. The five leave-one-out configurations are specified in Section 4.3.1 but

not yet implemented. No result is claimed for any of them anywhere in this chapter, and every corresponding table cell is shown as pending rather than filled with an estimate.

The baseline pipelines read the document through an identical sliding window that steps across the whole chapter with a uniform batch size, so no baseline is advantaged by seeing more or different content, and any difference between them reflects the prompting strategy alone.

Source data. Questions are generated from a college-level STEM document — the *Limits* chapter of OpenStax *Calculus, Volume 1* — ingested through the standard document pipeline (extraction → map-reduce summary → retrieval index). The extracted chapter text (about 112k characters) is also supplied to the judge as the ground-truth reference against which each question is scored.

Question count. Each of the two evaluated pipelines generated 200 MCQs spread across three target difficulties (easy n=68, medium n=66, hard n=66). The ablation and merging configurations are specified at n=100 per difficulty rather than 200: duplicate rate and grounding are large effects and separate at that size, and eleven configurations at 200 each is not affordable on a free-tier T4.

4.2 Evaluation Protocols

Generation and measurement are kept strictly separate: the Colab run only *generates* and saves every question to a single JSON artifact; standalone harnesses then read that artifact and score each question. Scoring an already-generated corpus with a separate program means the measurement can be repeated, or redone under a different protocol, without touching the system under test. It also means the ablation and merging corpora feed into an *identical* scoring path — the harnesses key on a `pipeline:difficulty` group label and are indifferent to which configuration produced a question.

Two published protocols are implemented, one harness each (`evaluate_eqgbench.py` and `evaluate_requesta.py`). Each is reported on *its own native scale*. No score is ever rescaled from one protocol onto another's range: EQGBench's three-point scale and ReQUESTA's four-point scale encode different judgements, and a linear remapping between them would produce a number that means nothing. In particular ReQUESTA's four-point scale is deliberately *even* — it removes the neutral midpoint that raters default to — whereas a five-point scale has one, so the two are not interconvertible even in principle.

4.2.1 Protocol A — EQGBench

EQGBench scores every question on five dimensions, each on a three-point scale $\{0, 1, 2\}$ (Poor / Good / Excellent):

- **KP — Knowledge-Point Alignment:** does the question accurately identify and reflect the knowledge point specified in the user's input?
- **QT — Question-Type Alignment:** does the item match the requested question type and adhere to that type's standard format?
- **QQ — Question Item Quality:** is the question expressed clearly, with an unambiguous objective and a definitive answer?
- **SQ — Solution Explanation Quality:** is the explanation correct, rigorous and complete, and does the answer follow from it?
- **CG — Competence-Oriented Guidance:** does the item integrate a realistic scenario that develops higher-order competence rather than rote recall?

Judge and voting. The judge is DeepSeek-R1 (`deepseek-reasoner`), the model EQG-Bench itself uses, at temperature 0.6 with a 4096-token output cap. Every question is judged **three times independently per dimension**; the reported score is the **mode** of the three rounds, and where no mode exists the **arithmetic mean** is used, following the benchmark. Each dimension is a *separate* judging call, because the published prompt is scoped to one dimension at a time.

Judge prompt. EQGBench publishes its evaluation prompt. The following is that prompt, reproduced verbatim, shown for the Knowledge-Point Alignment dimension:

```
System: You are an experienced middle school exam question designer with 20
years of expertise. Based on the following evaluation dimension, please
strictly score the given question according to the scoring criteria, in
combination with the user input and the generated question.

Query:
Dimension: Knowledge Point Alignment. This measures whether the generated
question accurately matches and adequately covers the specified knowledge
point.
Scoring Criteria:
0 points: The question item fails to correctly reflect the knowledge point.
  There is a significant mismatch between the knowledge point used in the
  question and the one specified by the user, or it is from a different
  subject.
1 point: The question item is generally relevant in topic but does not
  directly address or include the user-specified knowledge point.
2 points: The question item basically covers the user-specified knowledge
  point, with no significant omissions.

Example for 0 points:  [worked example with scoring justification]
Example for 2 points:  [worked example with scoring justification]

User Input: {user_input}
```

```
Generated Question: {generate_question}

Note: As long as the question includes the specified knowledge point in any
part, it is considered "basically covered" and earns 2 points. ...
Important: Only evaluate the content within the <question item> tags; ignore
<solution explanation> and <answer>. If the question is incomplete and cannot
stand alone, the score is automatically 0 for this dimension.

Please output the response in the following format:
[Scoring Justification]: ...<eoa>
[Score]: ...<eoa>
```

Two properties of this prompt are worth noting because they are easy to get wrong. First, the judge must state its *justification before* its score, which is a chain-of-thought ordering that materially affects the score. Second, the KP criterion is deliberately *lenient*: partial coverage anywhere in the item earns full marks. An evaluation that instructs the judge to "be strict" is therefore not measuring the same thing as EQGBench, and its scores are not comparable to the published figures.

Fidelity and deviations. The system prompt, query scaffold, KP criteria, both worked examples, the note block and the output format are taken verbatim from the benchmark. The QT, QQ, SQ and CG criteria are likewise verbatim. Three deviations are declared: (i) the published figure supplies worked examples for KP only, so the other four dimensions are zero-shot; (ii) the CG dimension defines only Excellent (2) and Poor (0) — there is *no* published one-point anchor, so the CG middle anchor used here is our own and CG is consequently the least comparable dimension; and (iii) EQGBench evaluates Chinese middle-school material against a real user query, whereas this work evaluates English calculus MCQs and reconstructs the user instruction from each question's topic and difficulty.

4.2.2 Protocol B — ReQUESTA

ReQUESTA's expert rubric scores two criteria dichotomously and the remainder on a four-point scale.

- **Binary** (0 = Disagree, 1 = Agree): *Answer correctness* — the correct answer is clearly correct to those who have comprehended the text; *Distractor incorrectness* — the distractors are clearly incorrect to those who have comprehended the text.
- **Four-point** (1 = Disagree, 2 = Neutral, 3 = Agree, 4 = Strongly Agree): overall quality, topic relevance, coverage, writing clarity, distractor plausibility, and distractor linguistic features.

An important limitation of this protocol. ReQUESTA publishes no judge prompt, and cannot: its rubric was applied by *two blinded human expert raters*, not

by a model. The raters calibrated on 100 practice items over three rounds and reached Cohen's weighted κ between .80 and 1.00. Consequently, what is reproduced here is the *rubric*, verbatim from the paper's appendix, together with the paper's disagreement rule — unresolved disagreements are settled by **taking the lower score**, a conservative rule that is the opposite of EQGBench's mean. The instruction wrapper that asks an LLM to apply that rubric is our own. An LLM applying a human rubric is a different instrument from two calibrated humans applying it, and no amount of prompt engineering closes that gap. Results under this protocol are reported with that caveat attached.

A documented inconsistency in the source rubric. The published appendix rubric and the published results tables do not list the same criteria. *Overall quality* and *coverage* are defined in the rubric but never reported; *distractor semantic uniqueness* is reported but has no anchor definition anywhere in the rubric. Our harness implements all of them and marks distractor semantic uniqueness as reconstructed from the paper's prose. Only the five criteria in Table 4.4 can be placed beside the published numbers at all.

4.2.3 Native metrics, and why a cost column is mandatory

Alongside the two published protocols, this project reports its own automated metrics: four judged quality dimensions on a 1–5 scale (`gemma3:12b` judging a `gemma3:4b` generator), plus automated measures in $[0, 1]$ — acceptance rate, judge-verified correctness, independent answer match, semantic grounding, duplicate rate, effective acceptance rate and inter-question similarity.

For the ablation and merging configurations these are joined by a **cost** measure: the number of LLM calls consumed per accepted question, recorded per question at generation time. This is not decoration. An ablation result of the form "removing agent X barely changed quality" is uninterpretable on its own; it becomes a finding only when read next to "and it removed a third of the calls". Equally, a merging result that preserves quality is worthless if it does not actually save compute. Every ablation and merging table in this chapter therefore carries a cost column, and no configuration is declared preferable on quality alone.

4.3 Ablation Design

4.3.1 Leave-one-agent-out

The agentic pipeline is not a monolith; it is five separable parts — the Quiz Planner, the model-directed retrieval tool, the Validator, the Refiner, and the cognitive-routing Controller. The leave-one-out family deletes exactly one of them per configuration, holding the other four and every hyper-parameter fixed (Table 4.1).

Two of these configurations deserve emphasis because they form a matched pair. The − validator condition removes the quality gate entirely: a draft is accepted as written. The

– refiner condition keeps the Validator, and keeps its verdict, but forbids it from acting — a failing question is labelled and retained rather than regenerated. Together they separate *detection* from *remediation*: the first asks whether noticing a bad question helps at all, the second whether acting on that verdict helps.

That pair is the sharpest instrument in this chapter, because it tests a specific mechanism this work has already proposed but never verified. Section 4.4.3 finds the agentic pipeline producing substantially more near-duplicate questions than the plain baseline, and Section 4.4.3 attributes that to the retry loop: a rejected question is regenerated against the *same* planned topic, so repeated retries concentrate the output on topics the planner has already chosen. If that explanation is correct, then – refiner — which retains validation but performs no retries — should show a duplicate rate markedly closer to the baseline's than to the full pipeline's. If it does not, the proposed mechanism is wrong and the excess repetition originates elsewhere, most plausibly in the planner. Either outcome is informative; at present the explanation is an untested hypothesis and is labelled as such.

4.3.2 Agent merging

Where the leave-one-out family asks *whether* an agent is needed, the merging family asks whether two agents need to be two *LLM calls*. The distinction matters because the agentic pipeline's cost is dominated by call count, not by any single agent's presence.

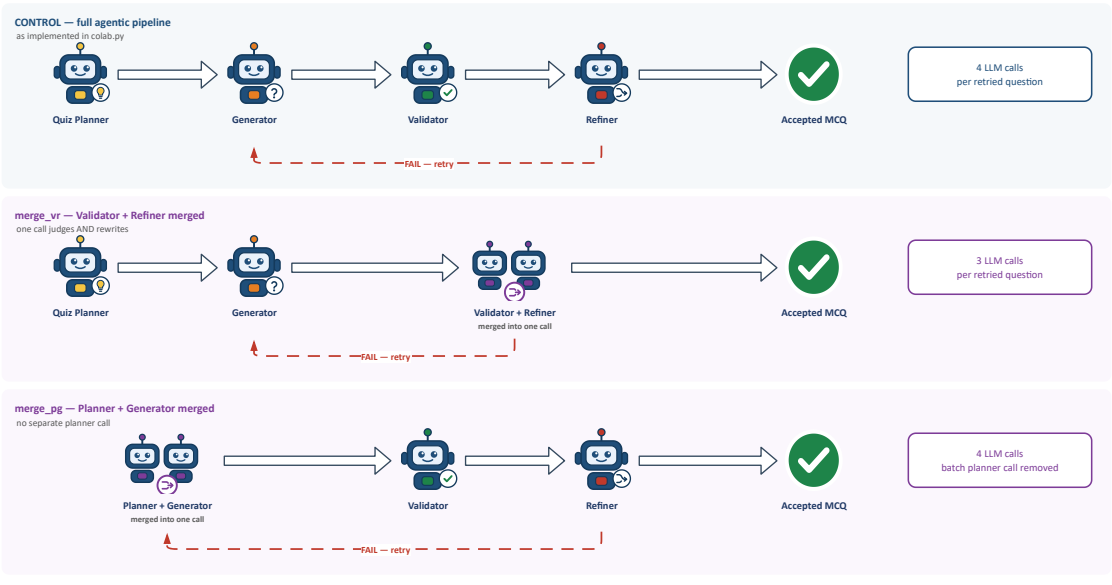

Figure 4.1: The three configurations compared by the merging experiment. All three receive identical grounded content and retrieval index and share the pinned `gemma3:4b` generator, the retrieval tool and the Controller; only the boxed agent merges differ. Call counts are structural — counted from the implementation, not measured runtimes.

`merge_vr` — Validator + Refiner. In the control pipeline a failing question costs two calls to fix: one to validate, and a second to redraft with the failure reason fed back. In

`merge_vr` the validator is instructed to repair what it rejects and to return the corrected question in the same reply, so a rejected question is re-validated directly rather than redrafted first.

Only the `regenerate` remediation can be absorbed this way. The other two actions the validator may choose — `fetch_context` and `replan_topic` — require work *outside* the model (a retrieval query, a topic swap) and are still returned as actions for the loop to act on, exactly as in the control. A merge cannot remove a step that was never an LLM call.

`merge_pg` — **Planner + Generator.** The control spends one planner call per batch and then one drafting call per question against an assigned topic. In `merge_pg` there is no standalone planner: each drafting call is shown the topics already used and must itself choose an unused topic that the summary genuinely covers, then write the question for it in the same reply. This trades a global view for a local one — the model can no longer balance topic counts across the whole quiz, only avoid what it has already seen. Whether that costs diversity is precisely what the experiment measures.

Structural call counts. Because both merges are changes to *packaging* rather than to behaviour, their cost can be counted from the implementation without running anything. For a question that is accepted on attempt k, and writing g for a drafting call and v for a validation call:

Table 4.2: LLM calls per question, counted structurally from the implementation. k is the number of attempts the question required. These are arithmetic, not measurements.

Configuration	**Calls for k attempts**	k=1	k=2
`agentic` (control)	$2k$ (+1 planner per batch)	2	4
`merge_vr`	$k+1$ (+1 planner per batch)	2	**3**
`merge_pg`	$2k$ (no batch planner call)	2	4

Two honest qualifications follow from this table. First, `merge_pg` saves nothing per question; its entire saving is the single batch-level planner call, which amortises to almost nothing over 100 questions. Its interest is therefore not cost but *diversity* — whether a generator that plans locally repeats itself more or less than one that plans globally. Second, `merge_vr`'s $k+1$ figure is realised *only when the merged call actually returns a usable rewrite.* When it returns a structurally invalid rewrite, or chooses `fetch_context` or `replan_topic`, the loop must redraft and the cost reverts to $2k$, the same as the control. The rate at which the merged call supplies a usable rewrite is thus itself a quantity to be measured, and the 25% saving at k=2 is an upper bound rather than a prediction.

4.3.3 What these ablations cannot show

Three limitations are inherent to the design and are stated here rather than discovered later. First, leave-one-out attributes an effect to the *absence* of one agent in the presence

of the other four; it cannot identify interactions between agents, for which a factorial design far beyond this project's compute budget would be required. Second, every quality number is produced by an LLM judge from the same model family as the generator, so an ablation that changes quality only slightly may be changing something the judge cannot see. Third, all configurations are evaluated on one chapter of one subject, so an agent that appears unnecessary on calculus may be necessary on material with different structure.

4.4 Results and Discussion

4.4.1 Protocol A results — EQGBench (scale 0–2)

Table 4.3 reports the EQGBench dimensions. All values in this table are on the benchmark's three-point scale. The reference columns are the figures published for the benchmark's own strongest judged models on the mathematics subset, and are included so that the magnitude of our scores can be read against a known anchor.

Table 4.3: EQGBench dimensions, all values on the 0–2 scale. Reference columns are published figures; the LectureAssist columns are from our own judging run under the verbatim protocol.

	Published reference		**LectureAssist**	
Dimension	**DeepSeek-R1**	**GPT-4o**	**Agentic**	**Non-agentic**
KP — Knowledge-Point Alignment	1.98	1.96	—	—
QT — Question-Type Alignment	1.95	1.82	—	—
QQ — Question Item Quality	1.95	1.76	—	—
SQ — Solution Explanation Quality	1.96	1.72	—	—
CG — Competence-Oriented Guidance†	0.17	0.21	—	—
Composite (mean of five, max 2)	1.40	1.29	—	—
Total (sum of five, max 10)	7.02	6.47	—	—
n questions	300	300	—	—

† CG carries a reconstructed middle anchor (see Section 4.2.1) and is the least comparable dimension.

Status. The LectureAssist columns are *not yet populated*. The evaluation harness was rebuilt to use the benchmark's verbatim published prompt, which supersedes an earlier run that used a reconstructed prompt instructing the judge to "be strict". Because that instruction contradicts the benchmark's own lenient scoring rule, scores from the earlier run are not comparable to the published figures and are not reported here. The re-run under the verbatim protocol is pending; at 5 dimensions × 3 rounds × 200 questions per pipeline it requires approximately 3,000 judge calls per pipeline.

What the published pattern predicts. The benchmark's headline finding is that KP, QT, QQ and SQ saturate near the ceiling (1.9–2.0) for essentially every capable model and therefore do not discriminate between systems, while CG collapses for all of them — the best mathematics score across 46 evaluated models does not exceed 0.31. The discriminating dimension is the demanding one. This is the same pattern this project

observes with its own metrics, where acceptance rate saturates and duplicate rate carries the signal (Section 4.4.3), and it is the reason both are reported. It also has a direct consequence for the ablations: they should be read on duplicate rate, grounding and cost, not on the judged dimensions, which have no headroom left to move.

4.4.2 Protocol B results — ReQUESTA (scale 1–4 and 0–1)

Table 4.4 reports the ReQUESTA rubric. The four-point criteria are on the 1–4 scale and the binary criteria on 0–1; the two groups are separated in the table and are never averaged together.

Table 4.4: ReQUESTA expert-rubric criteria. Four-point criteria on the 1–4 scale; binary criteria on the 0–1 scale. Reference columns are the published expert-rated means (N=200).

	Published (human experts)		**LectureAssist (LLM judge)**	
Criterion	**ReQUESTA**	**GPT-5**	**Agentic**	**Non-agentic**
Four-point criteria (1–4)				
Topic relevance	3.36	3.12	—	—
Writing clarity	3.02	**3.39**	—	—
Distractor linguistic features	2.28	1.92	—	—
Distractor semantic plausibility	3.17	2.63	—	—
Distractor semantic uniqueness‡	3.82	3.75	—	—
Overall quality	n/r	n/r	—	—
Coverage	n/r	n/r	—	—
Binary criteria (0–1)				
Answer correctness	1.00	1.00	—	—
Distractor incorrectness	0.99	0.98	—	—

‡ Reported in the source paper but absent from its appendix rubric; anchors reconstructed.
n/r = defined in the rubric but never reported in the source paper.

Status. As with Protocol A, the LectureAssist columns are *not yet populated*; the harness is implemented and verified but has not been run. The published columns are reproduced here because they establish the comparison target.

The rigour-versus-clarity trade-off. The published result worth attention is the reversal on *writing clarity*: the agentic system beats the single-pass GPT-5 baseline on topic relevance, distractor linguistic features and distractor semantic plausibility, but *loses* to it on writing clarity (3.02 versus 3.39, $p < .001$). Questions that demand integration and inference need richer contextual framing, which costs brevity. This is a directly testable prediction for our own agentic pipeline, and it is the single most useful hypothesis either protocol supplies.

What this protocol cannot deliver here. The source study also reports item difficulty (p), discrimination (D) and point-biserial correlation, obtained from 572 human participants answering the items. Those are psychometric properties of *learner responses*,

not of question text, and no LLM judge can substitute for them. They are outside the scope of this work and are listed as a limitation.

4.4.3 Native metrics — agentic versus non-agentic

Table 4.5 reports this project's own automated metrics for the two configurations that have completed a scored run. These are on a five-point scale for the judged quality dimensions and on a 0–1 rate scale for the automated measures; they are reported separately from both published protocols and are never mixed with them. The judge is `gemma3:12b`, generating from a `gemma3:4b` generator.

Table 4.5: Native metrics, n=200 questions per pipeline. Judged dimensions are on a 1–5 scale; the remaining measures are rates or cosine scores in $[0, 1]$. Higher is better for every row *except* duplicate rate and inter-question similarity.

Metric	Scale	Agentic	Non-agentic
Overall judged quality	1–5	4.86	**4.93**
Correctness	1–5	4.98	**5.00**
Clarity	1–5	4.92	**4.99**
Difficulty match	1–5	4.36	**4.39**
Acceptance rate	0–1	0.995	**1.000**
Judge-verified correctness	0–1	0.995	0.995
Independent answer match	0–1	0.940	**0.970**
Semantic grounding	0–1	0.557	**0.624**
Duplicate rate (↓)	0–1	0.385	**0.280**
Effective acceptance rate	0–1	0.612	**0.720**
Inter-question similarity (↓)	0–1	0.932	**0.894**

Table 4.6: Native metrics broken down by target difficulty, n per cell as shown. Higher is better except duplicate rate.

Difficulty	Pipeline	n	Duplicate	Grounding	Eff. accept	Indep. match	Verified
Easy	Agentic	68	0.353	0.562	0.647	0.971	1.000
	Non-agentic	68	0.324	0.642	0.676	0.971	0.985
Medium	Agentic	66	0.379	0.595	0.612	0.879	0.985
	Non-agentic	66	0.227	0.614	0.773	0.955	1.000
Hard	Agentic	66	0.288	0.513	0.712	0.970	1.000
	Non-agentic	66	0.182	0.616	0.818	0.985	1.000

Reading these results honestly. On this instrument the agentic pipeline does **not** outperform the non-agentic baseline. Non-agentic matches or beats agentic on every judged dimension, produces fewer duplicates (0.280 versus 0.385), achieves a higher effective acceptance rate (0.720 versus 0.612), and is better grounded in the source text (0.624 versus 0.557). The pattern holds at every difficulty level.

Two observations follow. First, the four judged quality dimensions *saturate*: every value sits between 4.36 and 5.00 on a five-point scale, and correctness reaches the ceiling exactly. A metric that cannot go up cannot discriminate, which is precisely the failure mode EQGBench documents for its own four saturating dimensions. The dimensions that

do separate the two pipelines are the automated ones — duplicate rate, grounding and effective acceptance — and all three favour the baseline.

Second, the plausible mechanism is the agentic loop's retry behaviour. Because a rejected question is regenerated against the same planned topic, repeated retries concentrate the output around topics the planner has already selected, raising duplication. The validator enforces per-question quality but has no view of corpus-level diversity. **This remains a hypothesis.** It is consistent with the data but is not established by it, and it is exactly what the − refiner and − planner ablations of Section 4.3.1 are designed to test. On the present evidence the agentic pipeline's advantage over single-pass prompting is **not established**.

4.4.4 Leave-one-agent-out results

Table 4.7 is the reporting frame for the leave-one-out family. It is restricted to the measures that Section 4.4.3 shows actually discriminate, plus cost; the judged 1–5 dimensions are omitted because they saturate and would fill the table with values that cannot move.

Table 4.7: Leave-one-agent-out ablation, planned at n=100 per difficulty. Higher is better except duplicate rate and calls. Δ columns are relative to the full agentic control. No cell is populated: these configurations are specified but not yet implemented.

Configuration	**Duplicate** (↓)	**Grounding**	**Eff. accept**	**Indep. match**	**Calls / accepted** (↓)
`agentic` (control)	0.385*	0.557*	0.612*	0.940*	—
− planner	—	—	—	—	—
− retrieval	—	—	—	—	—
− validator	—	—	—	—	—
− refiner	—	—	—	—	—
− routing	—	—	—	—	—
`nonagentic` (all removed)	0.280*	0.624*	0.720*	0.970*	—

* Carried over from Table 4.5 at n=200, not n=100; the two control rows are therefore not sample-size-matched to the ablation rows and will be re-run at n=100 before any Δ is computed. Cost is shown as pending for the control as well, because call counts were not instrumented during the original run — the instrumentation was added with the ablation harness.

Status. Not measured. The five configurations are specified in Section 4.3.1; the flags that select them are not yet implemented, and no number will be entered into this table until they are run.

The prediction on record. Stating a prediction before the run is what makes the result falsifiable. If the retry-concentration mechanism of Section 4.4.3 is correct, then − refiner should show a duplicate rate substantially below the control's 0.385 and closer to the baseline's 0.280, while − validator — which also removes retries — should behave similarly on duplication but worse on grounding, since nothing then blocks an ungrounded question. If instead − planner is the configuration that drops duplication, the cause is topic

selection rather than retrying, and the mechanism stated in Section 4.4.3 is wrong and should be corrected in this chapter rather than defended.

4.4.5 Agent-merging results

Table 4.8 is the reporting frame for the merging family. The two merged configurations are implemented and verified in `merged_agents.py`, which reuses the control pipeline's prompts verbatim so that the merge is the only difference, and which emits the same artifact the two published harnesses already read.

Table 4.8: Agent-merging variants, planned at n=100 per difficulty. Higher is better except duplicate rate and calls. The structural call counts of Table 4.2 are repeated for reference; the measured columns are not populated, as neither configuration has been run.

Configuration	**Structural calls at k=2**	**Duplicate** (↓)	**Grounding**	**Eff. accept**	**Calls / accepted** (↓) **(measured)**
`agentic` (control)	4	0.385*	0.557*	0.612*	—
`merge_vr`	3	—	—	—	—
`merge_pg`	4	—	—	—	—

* As in Table 4.7, carried over at n=200 and not sample-size-matched.

Status. Not measured. The harness is implemented and its control flow verified against a stubbed model — confirming, among other things, that a retried question costs three calls rather than four under `merge_vr`, and that a question whose retry budget is exhausted is recorded as an explicit override rather than a silent pass — but no scored generation run has been performed.

The prediction on record. `merge_vr` should reduce measured calls per accepted question relative to the control, by an amount bounded above by 25% at k=2 and realised only in proportion to how often the merged call returns a usable rewrite. Its risk is quality: a model asked to judge and rewrite in one reply may become a lenient judge of its own rewrite, which would show up as a higher acceptance rate paired with unchanged or worse grounding — a pattern worth watching for specifically, since acceptance rate alone would look like an improvement. `merge_pg` should be approximately cost-neutral and is being run for its effect on duplication, where losing the planner's global view could plausibly move the metric in either direction.

4.5 Summary

This chapter evaluated question-generation pipelines built on a pinned local `gemma3:4b` generator under three instruments reported on their own scales: the EQGBench protocol (five dimensions, 0–2, DeepSeek-R1 judge, three-round voting with an arithmetic-mean tie-break), the ReQUESTA expert rubric (two binary criteria and six four-point criteria, disagreements resolved downward), and this project's native metrics (1–5 judged dimensions plus automated rates and a per-question call count).

The evaluation deliberately separates generation from measurement so that a corpus can be rejudged without re-running the generator, and so that eleven configurations can be scored through one identical path. Both published protocols are implemented with their rubrics transcribed verbatim from source, and every deviation is declared rather than absorbed silently — including the absence of any published judge prompt for ReQUESTA, whose rubric was applied by human experts.

Of the eleven configurations, two have completed a scored run. On the native metrics the non-agentic baseline currently outperforms the agentic pipeline, driven by lower duplication and better grounding, while the judged quality dimensions saturate and do not discriminate. That negative result is what motivates the two ablation families introduced in this chapter: the leave-one-out configurations exist to identify *which* agent produces the excess duplication, and the merging configurations exist to establish what the remaining agents cost. The retry-concentration explanation offered for the negative result is, at present, an untested hypothesis with a stated, falsifiable prediction attached to it.

Results under the two published protocols are pending their judging runs; the CoT and PoT pipelines remain unevaluated; the merging harness is implemented but unrun; and the leave-one-out configurations are specified but not implemented. No claim is made for any of them until those runs complete.